Assignment 6: Implementation of Data Integrity Constraints

DBMS Assignment Series | Project topic: **Online Food Delivery System (QuickBite)**

Name: **Jagan S** | Roll No.: **2501121240031**

1. Objective

Implement primary keys, foreign keys, UNIQUE, CHECK and DEFAULT constraints on the QuickBite tables, explain why each is used, and run SQL statements that prove integrity and relationships are enforced.

2. Constraints implemented (all declared in the CREATE TABLE statements of Assignment 5)

Constraint	Where used	Purpose
PRIMARY KEY	Every table: customer_id, restaurant_id, partner_id, item_id, order_id, order_item_id	Uniquely identifies each row; cannot be NULL or duplicated.
FOREIGN KEY	menu_items.restaurant_id; orders.customer_id, restaurant_id, partner_id; order_items.order_id, item_id	Referential integrity: a child row can only point to an existing parent row. RESTRICT stops deleting a customer/restaurant/dish that is in use; CASCADE on order_items removes the lines of a deleted order; SET NULL on orders.partner_id keeps the order when a partner leaves.
UNIQUE	customers.email, phone; delivery_partners.phone; restaurants(name, area); menu_items(restaurant_id, item_name); order_items(order_id, item_id)	Prevents duplicate accounts, duplicate phone numbers, the same restaurant or dish listed twice, and the same dish repeated as two lines of one order.
CHECK	menu_items.price > 0; restaurants.rating BETWEEN 0 AND 5; order_items.quantity BETWEEN 1 AND 20; unit_price > 0; delivery_fee >= 0; status, payment_method, membership, vehicle, availability, category value lists; phone length = 10; e-mail format	Business rules enforced by the database itself - invalid values are rejected no matter which app inserts them.
DEFAULT	customers.area = 'Koramangala', membership = 'Regular', signup_date = CURRENT_DATE; orders.status = 'Placed', payment_method = 'UPI', delivery_fee = 30, order_date = CURRENT_DATE; menu_items.is_veg = 1; delivery_partners.vehicle = 'Bike', availability = 'Available'	Supplies a sensible value when none is given, simplifying INSERT statements and reducing errors.
NOT NULL	Mandatory columns such as names, phone, price, quantity	Ensures required information is always present.

Note: Foreign-key checking must be switched on in SQLite with `PRAGMA foreign_keys = ON;` (it is always on in MySQL InnoDB / PostgreSQL). It was enabled for every test below.

3. Verification - statements and their actual outputs

A small starting data set is loaded first; then valid statements (should succeed) and invalid statements (should be rejected) are executed.

3.0 Seed data (succeeds)

```
INSERT INTO customers (full_name, email, phone, area, signup_date, membership)
VALUES ('Aditya Rao', 'aditya.rao@example.com', '9811100001', 'Indiranagar', '2026-01-10', 'Regular');
INSERT INTO restaurants (restaurant_name, cuisine, area, rating, is_open)
VALUES ('Spice Garden', 'North Indian', 'Indiranagar', 4.4, 1);
INSERT INTO delivery_partners (partner_name, phone, vehicle, availability)
VALUES ('Ravi Kumar', '9822200001', 'Bike', 'Available');
INSERT INTO menu_items (restaurant_id, item_name, category, price, is_veg)
VALUES (1, 'Paneer Tikka', 'Starter', 220, 1), (1, 'Butter Chicken', 'Main Course', 320, 0);
INSERT INTO orders (customer_id, restaurant_id, partner_id, order_date, status, payment_method, delivery_fee)
VALUES (1, 1, 1, '2026-10-01', 'Delivered', 'UPI', 30);
INSERT INTO order_items (order_id, item_id, quantity, unit_price) VALUES (1, 1, 2, 220);
```

3.1 DEFAULT constraint: Insert a customer with only name, e-mail and phone; area and membership are filled automatically.

```
INSERT INTO customers (full_name, email, phone) VALUES ('Test User', 'test.user@example.com', '9811199999');

SELECT full_name, area, membership FROM customers WHERE email = 'test.user@example.com';
```

Result: Success - 1 row(s) affected

full_name	area	membership
Test User	Koramangala	Regular

3.2 DEFAULT constraint: Insert an order giving only customer and restaurant; status, payment method and delivery fee default.

```
INSERT INTO orders (customer_id, restaurant_id) VALUES (1, 1);

SELECT order_id, status, payment_method, delivery_fee FROM orders WHERE order_id = 2;
```

Result: Success - 1 row(s) affected

order_id	status	payment_method	delivery_fee
2	Placed	UPI	30

3.3 UNIQUE constraint: Duplicate e-mail address is rejected.

```
INSERT INTO customers (full_name, email, phone) VALUES ('Another Person', 'aditya.rao@example.com', '9811100077');
```

Result: REJECTED: UNIQUE constraint failed: customers.email

3.4 UNIQUE constraint: Duplicate phone number is rejected.

```
INSERT INTO customers (full_name, email, phone) VALUES ('Another Person', 'other@example.com', '9811100001');
```

Result: REJECTED: UNIQUE constraint failed: customers.phone

3.5 UNIQUE constraint: The same dish listed twice for one restaurant is rejected.

```
INSERT INTO menu_items (restaurant_id, item_name, category, price) VALUES (1, 'Paneer Tikka', 'Starter', 230);
```

Result: REJECTED: UNIQUE constraint failed: menu_items.restaurant_id, menu_items.item_name

3.6 CHECK constraint: Negative price is rejected.

```
INSERT INTO menu_items (restaurant_id, item_name, category, price) VALUES (1, 'Free Lunch', 'Main Course', -50);
```

Result: REJECTED: CHECK constraint failed: price > 0

3.7 CHECK constraint: Rating above 5 is rejected.

```
INSERT INTO restaurants (restaurant_name, cuisine, area, rating) VALUES ('Bad Rating', 'Thai', 'HSR Layout', 6);
```

Result: REJECTED: CHECK constraint failed: rating BETWEEN 0 AND 5

3.8 CHECK constraint: Quantity of 0 is rejected.

```
INSERT INTO order_items (order_id, item_id, quantity, unit_price) VALUES (2, 2, 0, 320);
```

Result: REJECTED: CHECK constraint failed: quantity BETWEEN 1 AND 20

3.9 CHECK constraint: Invalid order status is rejected.

```
UPDATE orders SET status = 'Shipped' WHERE order_id = 1;
```

Result: REJECTED: CHECK constraint failed: status IN ('Placed', 'Delivered', 'Cancelled')

3.10 FOREIGN KEY: Menu item for a non-existent restaurant is rejected.

```
INSERT INTO menu_items (restaurant_id, item_name, category, price) VALUES (99, 'Ghost Dish', 'Starter', 100);
```

Result: REJECTED: FOREIGN KEY constraint failed

3.11 FOREIGN KEY: Order for a non-existent customer is rejected.

```
INSERT INTO orders (customer_id, restaurant_id) VALUES (99, 1);
```

Result: REJECTED: FOREIGN KEY constraint failed

3.12 FOREIGN KEY (RESTRICT): Deleting a customer who has orders is rejected.

```
DELETE FROM customers WHERE customer_id = 1;
```

Result: REJECTED: FOREIGN KEY constraint failed

3.13 FOREIGN KEY (RESTRICT): Deleting a restaurant that still has menu items is rejected.

```
DELETE FROM restaurants WHERE restaurant_id = 1;
```

Result: REJECTED: FOREIGN KEY constraint failed

3.14 FOREIGN KEY (SET NULL): Deleting a delivery partner keeps the order but clears its partner_id.

```
DELETE FROM delivery_partners WHERE partner_id = 1;

SELECT order_id, status, partner_id FROM orders WHERE order_id = 1;
```

Result: Success - 1 row(s) affected

order_id	status	partner_id
1	Delivered	NULL

3.15 FOREIGN KEY (CASCADE): Deleting an order automatically deletes its order lines.

```
DELETE FROM orders WHERE order_id = 1;

SELECT COUNT(*) AS order_lines_remaining FROM order_items;
```

Result: Success - 1 row(s) affected

order_lines_remaining
0

3.16 Referential-integrity audit

```
PRAGMA foreign_key_check;   -- lists every row that violates a foreign key
```

Output: (no rows returned) - no orphan records exist; all relationships between tables are intact.

4. Conclusion

Every valid statement was accepted and every invalid statement (duplicates, negative values, bad domain values, broken references) was rejected by the DBMS with a constraint error. This confirms that data integrity and the relationships between the tables are maintained automatically.

Note: Error wording differs slightly between SQLite and MySQL (e.g. MySQL reports error 1062 for duplicates and 1452 for foreign-key failures) but the behaviour is the same.